



Annexe 1 — RMI : workflow et aide graduée

Année 2026–2027 Enseignant Bachar Rima

À QUOI SERT CETTE ANNEXE

Elle rassemble ce dont on a besoin *pendant* le TP : l'ordre des opérations, les décisions à prendre, les erreurs que l'on rencontre réellement et la façon de les lire.

1. Le modèle en une page

ACTEUR	CE QUE C'EST	OÙ IL VIT
Interface distante	une interface Java qui étend <code>java.rmi.Remote</code> ; chaque méthode déclare <code>RemoteException</code>	projet <code>common</code> — connue des deux côtés
Servant	la classe d'implémentation, en général héritant de <code>UnicastRemoteObject</code>	projet <code>server</code> — jamais chez le client
Stub	le représentant local du servant, engendré à l'exportation	fabriqué par le runtime, obtenu par <code>lookup</code>
Registre	un annuaire : associe un nom à un stub	processus <code>rmiregistry</code> , ou créé dans la JVM du serveur
Objet sérialisable	une donnée transmise <i>par valeur</i>	projet <code>common</code> — la classe doit être connue des deux côtés

LA SEULE QUESTION À SE POSER POUR CHAQUE TYPE

Pour chaque paramètre, chaque valeur de retour, chaque attribut exposé :

« Est-ce que je veux envoyer une *photo* de l'objet, ou un *moyen de le joindre*? »

- Une photo → la classe implémente `java.io.Serializable`. L'objet est copié; il y a désormais **deux objets distincts**.
- Un moyen de le joindre → la classe implémente une interface `Remote` et l'objet est exporté. C'est le **stub** qui est transmis; les appels reviennent toujours à l'objet d'origine.
- Ni l'un ni l'autre → l'appel échoue à l'empaquetage, avec une `NotSerializableException` encapsulée dans une `MarshalException`.

DEUX FORMULATIONS À NE PAS REPRENDRE

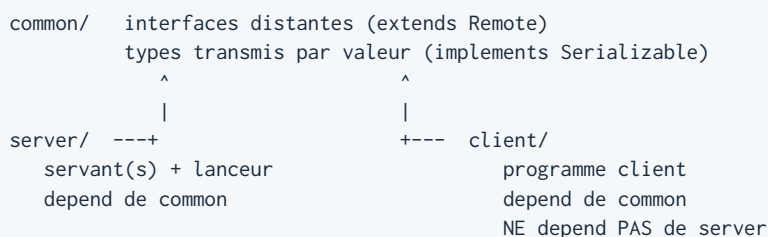
On lit souvent que « *tous les objets Java sont passés par référence* ». C'est faux, et c'est une source durable de confusion.

Java passe toujours par valeur. Pour une variable de type objet, la valeur passée est une *référence* vers l'objet : l'appelé peut modifier l'objet pointé, mais réaffecter le paramètre n'a aucun effet chez l'appelant.

En RMI, il faut une formulation encore plus précise : un objet `Remote` exporté est représenté chez le destinataire par un **stub**, tandis qu'un objet `Serializable` non `Remote` est **sérialisé et copié**. C'est cette distinction-là que le TP vous demande de manipuler.

De même, on lit parfois que la sérialisation produit « une représentation textuelle ». La sérialisation native de Java produit un **flux binaire** d'objets : *état de l'objet* → *suite d'octets*, puis *suite d'octets* → *objet reconstruit*. Et la copie reconstruite n'a jamais la même identité que l'original.

2. Structure des projets



Le test qui ne trompe pas. Supprimez (mentalement) le projet `server` du `classpath` de votre client : celui-ci doit toujours compiler. S'il ne compile plus, c'est que le client référence une classe d'implémentation — c'est un défaut de conception, pas un détail.

METTRE COMMON SUR LE CLASSPATH

- **Eclipse** — clic droit sur `server` (puis `client`) → *Build Path* → *Configure Build Path...* → onglet *Projects* → *Add...* → `common`.
- **IntelliJ IDEA** — *File* → *Project Structure* → *Modules* → `server` → *Dependencies* → + → *Module Dependency* → `common`.
- **Ligne de commande** — `javac -cp common/out -d server/out ...` puis `java -cp common/out:server/out ...` (; au lieu de : sous Windows).

NE CRÉEZ PAS DE PROJET JAVA MODULAIRE

À la création d'un projet, Eclipse propose d'engendrer un `module-info.java`. **Ne le faites pas** pour ce TP : vous devriez alors déclarer explicitement `requires java.rmi` et ouvrir vos paquets, ce qui n'apprend rien sur RMI et coûte du temps.

Ce n'est pas une limitation de RMI : `java.rmi` est un module parfaitement utilisable depuis une application modulaire. C'est un choix de simplicité pédagogique. Ce TP se fait sur le `classpath`.

3. Workflow

3.1. Écrire un objet distant

1. Le contrat, dans `common` :

```
1 public interface IMonService extends Remote {
2     TypeRetour maMethode(Parametres p) throws RemoteException;
3 }
```

Chaque méthode **doit** déclarer `RemoteException` : c'est le compilateur qui vous force à prévoir la défaillance partielle.

2. Le servant, dans `server` :

```
1 public class MonServiceImpl extends UnicastRemoteObject
2                               implements IMonService {
3
4     public MonServiceImpl() throws RemoteException {
5         super();           // c'est ici que l'objet est exporte
6     }
7
8     @Override
9     public TypeRetour maMethode(Parametres p) throws RemoteException {
10         // ...
11     }
12 }
```

Le constructeur déclare `RemoteException` parce que `super()` exporte l'objet, et que l'export peut échouer.

3. Variante : si votre classe doit hériter d'une autre classe, ne l'étendez pas et exportez-la explicitement — `UnicastRemoteObject.exportObject(this, 0)`. Les deux approches sont valides (CM 02, section 2).

ATTENTION

Une classe qui étend seulement `UnicastRemoteObject` sans implémenter d'interface distante n'est pas un objet distant utilisable.

3.2. Publier : le serveur

Le serveur fait trois choses, dans cet ordre : il obtient un registre, il crée le ou les servants, il les lie à des noms.

```
1 // A. registre lance a part, dans son propre processus (commande rmiregistry)
2 Registry registry = LocateRegistry.getRegistry();           // ou (port)
3
4 // B. OU BIEN registre cree dans la JVM du serveur
5 Registry registry = LocateRegistry.createRegistry(1099);
6
7 IMonService service = new MonServiceImpl();
8 registry.rebind("MonService", service);
```

	GETREGISTRY()	CREATEREGISTRY(1099)
Effet	obtient une <i>référence</i> vers un registre supposé existant	crée et met en écoute un registre dans la JVM courante
Processus	3 : <code>rmiregistry</code> , serveur, client	2 : serveur, client
Durée de vie	indépendante du serveur	meurt avec le serveur
Échec typique	<code>ConnectException</code> au <code>bind</code>	<code>ExportException: Port already in use</code>

LE PIÈGE DE GETREGISTRY

`LocateRegistry.getRegistry()` **ne contacte pas** le registre et ne retourne **jamais** `null`. Un test `if (registry == null)` est du code mort : l'absence de registre n'est découverte qu'au premier appel réel, c'est-à-dire au `bind` (TD 1, question 6(b)).

bind ou rebind? `bind` refuse d'écraser une association existante et lève `AlreadyBoundException`; `rebind` remplace silencieusement. Pendant le développement, `rebind` évite de devoir redémarrer le registre à chaque relance du serveur. En exploitation, `bind` rend visible une double publication au lieu de la masquer.

NOTE

Les mêmes opérations existent sous forme d'URL avec `java.rmi.Naming` :
`Naming.rebind("rmi://localhost:1099/MonService", service).`
C'est une commodité bâtie au-dessus de `Registry`; les deux styles coexistent et font la même chose.

3.3. Trouver et invoquer : le client

```
1 Registry registry = LocateRegistry.getRegistry(hote, 1099);
2 IMonService service = (IMonService) registry.lookup("MonService");
3 service.maMethode(parametres);
```

- Le premier argument est un **nom d'hôte**, pas un numéro de port. `null` signifie « hôte local ».
- Le transtypage vise **l'interface**, jamais la classe d'implémentation. Écrire `(MonServiceImpl) registry.lookup(...)` compile si la classe est sur le classpath, mais lève une `ClassCastException` à l'exécution : le stub implémente l'interface, il n'hérite pas de la classe.

3.4. Ordre de lancement

- | | |
|--------------------|--|
| 1. le registre | (rmiregistry, OU createRegistry dans le serveur) |
| 2. le serveur | (il publie : bind / rebind) |
| 3. le(s) client(s) | (ils cherchent : lookup) |

Lancer le client avant le serveur donne une `NotBoundException`; lancer le serveur sans registre donne une `ConnectException`. Ce sont les symptômes normaux du mauvais ordre.

3.5. Faire rappeler le client par le serveur

Pour l'étape A6 du TP1, le client doit exporter à son tour un objet distant et en confier le stub au serveur.

- L'interface de l'observateur va dans `common` : les **deux** côtés doivent la connaître.
- L'implémentation va chez le client et doit être **exportée** (héritage de `UnicastRemoteObject`, ou `exportObject`).
- Le serveur la reçoit en paramètre : il en obtient un stub, exactement comme le client en obtient un du registre.
- « Client » et « serveur » sont des **rôles dans une interaction**, pas des identités de machines. Un même processus peut tenir les deux.

ROBUSTESSE DES NOTIFICATIONS

Un observateur peut avoir disparu depuis son abonnement. Notifier dans une boucle sans protection fera échouer toute la notification au premier client mort. Attrapez la `RemoteException` par observateur, et décidez quoi faire de celui qui ne répond plus. Pensez aussi à ce qui se passe si un client s'abonne pendant qu'une notification est en cours.

3.6. Du scénario de test à l'application utilisable

Pendant le développement, un `main` qui enchaîne quelques appels est le moyen le plus rapide de vérifier qu'une étape fonctionne. Mais ce n'est pas une application : pour l'utiliser autrement, il faut modifier le code.

FORME	RÔLE	STATUT DANS LE TP 1
appels dans <code>main</code>	tester une étape, déboguer	outil de développement
interface en ligne de commande	rendre l'application utilisable	obligatoire (étape A8)
interface graphique	même rôle, autre présentation	bonus (partie C)

Séparer trois responsabilités. Si la lecture du clavier, l’affichage et les appels distants sont mélangés dans un seul `main`, le code devient vite illisible — et une interface graphique obligerait à tout réécrire.

```

point d'entree    arguments (hote, port), lookup, lancement de l'interface, arret
|
interface        menu, saisies, affichages                <- aucun java.rmi ici
|
logique client   operations sur les interfaces distantes,
                  RemoteException traduite en message pour l'utilisateur
|
stub  ----->   serveur

```

POINTS TECHNIQUES D'UNE CLI CLIENTE

- **Lire les saisies** : `Scanner` ou `BufferedReader` suffisent. `BufferedReader.readLine()` renvoie `null` en fin d’entrée (Ctrl+D, Ctrl+Z) : traitez-le comme une demande de sortie.
- **Valider** : un choix de menu inconnu, un champ vide ou un nombre mal formé doivent produire un message et une nouvelle saisie, pas une exception.
- **Afficher les alertes** : elles arrivent sur un fil d’exécution du runtime RMI, à tout moment, y compris pendant une saisie. Un préfixe reconnaissable suffit à les distinguer.
- **S’arrêter vraiment** : un objet exporté — votre observateur — garde la JVM en vie. Avant de quitter, désabonnez-vous si possible et désexportez-le avec `UnicastRemoteObject.unexportObject(objet, true)`.
- **Survivre à une panne du serveur** : une `RemoteException` pendant une opération est une situation normale en réparti ; elle se signale à l’utilisateur, elle n’arrête pas le client.

4. Erreurs fréquentes, et comment les lire

MESSAGE	CAUSE HABITUELLE	CE QU'IL FAUT FAIRE
<code>ConnectException:</code> <code>Connection refused</code> <code>NotBoundException</code>	aucun registre n’écoute sur le port visé le nom cherché n’est pas lié — client lancé avant le serveur, ou faute de frappe dans le nom	lancer <code>rmiregistry</code> , ou passer à <code>createRegistry</code> vérifier l’ordre de lancement et l’orthographe exacte
<code>AlreadyBoundException</code>	un serveur tourne déjà et a publié ce nom	arrêter l’instance précédente, ou utiliser <code>rebind</code>
<code>ExportException:</code> <code>Port already in use</code>	<code>createRegistry</code> sur un port déjà occupé	arrêter l’autre processus ; vérifier qu’aucune exécution précédente ne traîne
<code>ClassNotFoundException</code> ... (no security manager: RMI class loader disabled)	le destinataire ne connaît pas la classe de l’objet reçu	placer le type partagé dans <code>common</code> (voir § 5)
<code>NotSerializableException</code> dans une <code>MarshalException</code>	un paramètre n’est ni <code>Remote</code> ni <code>Serializable</code>	choisir la sémantique voulue et implémenter l’interface correspondante
<code>ClassCastException</code> après lookup	transtypage vers la classe d’implémentation	transtyper vers l’ interface
<code>java.rmi.ConnectIOException:</code> ... connection to: 1099	un numéro de port a été passé là où un nom d’hôte est attendu	<code>getRegistry(hote, port)</code> prend les deux séparément

MÉTHODE DE LECTURE D'UNE EXCEPTION RMI

Les exceptions RMI sont **emboîtées**. Lisez-les de l’intérieur vers l’extérieur : la couche externe dit *où* cela a cassé (`ServerException` = côté serveur), la **cause la plus profonde** dit *quoi*. En Java : parcourez `getCause()` jusqu’au bout avant de chercher sur le Web.

DEUX RÉFLEXES QUI FONT GAGNER DU TEMPS

- **Un processus oublié.** Un serveur d'une exécution précédente occupe encore le port. Sous Eclipse, la vue *Console* permet de vérifier qu'aucun programme n'est encore actif (bouton rouge d'arrêt); en ligne de commande, fermez le terminal ou terminez le processus.
- **Le registre ne voit pas vos classes.** Le registre déséréalise le stub qu'on lui confie : il doit pouvoir charger votre interface distante. Lancez `rmiregistry` depuis le répertoire des classes compilées de `common`, ou avec la variable `CLASSPATH` positionnée dessus.

5. Ce que vous lirez ailleurs et qui n'est plus d'actualité

Beaucoup de tutoriels RMI encore en ligne ont été écrits avant Java 9. Ils décrivent des mécanismes qui ont depuis été retirés du JDK ou neutralisés par défaut.

MÉCANISMES RETIRÉS OU NEUTRALISÉS

MÉCANISME	STATUT	AUJOURD'HUI
<code>rmic</code> · stubs et squelettes engendrés	inutile depuis Java 5, retiré du JDK 15	le stub est un proxy dynamique créé à l'export
<code>SecurityManager</code> · <code>.policy</code>	déprécié pour suppression (JEP 411, Java 17); lève <code>UnsupportedOperationException</code> dès Java 18; désactivé définitivement en Java 24 (JEP 486)	inutile dans le cas nominal : n'en installez pas
<code>java.rmi.server.codebase</code>	la propriété existe, mais <code>useCodebaseOnly</code> vaut <code>true</code> par défaut depuis Java 7u21, et le chargeur RMI est désactivé sans <code>SecurityManager</code>	partager le type via le projet <code>common</code>
<code>java.rmi.activation</code> · <code>rmid</code>	retiré du JDK 17 (JEP 407)	un serveur reste lancé
Tunnel HTTP	déprécié et désactivé par défaut depuis Java 8	VPN ou TLS

Ce qui reste vrai, et que le TP vous fait observer : Un flux sérialisé contient l'état de l'objet et le **nom** de sa classe. Il ne contient **pas** son bytecode. Le destinataire doit donc déjà disposer de la définition de classe pour reconstruire l'objet — d'où le message `ClassNotFoundException`. Le `codebase` était une réponse à ce problème; la réponse actuelle est de **placer le type partagé dans `common`**, ce qui est plus simple, plus sûr, et fonctionne sur toutes les versions de Java.

6. Bonnes pratiques de conception

- **Encapsulation** — attributs `private`, accès par méthodes. `protected` seulement si les sous-classes en ont réellement besoin.
- **Nommage** — `camelCase` pour variables et méthodes, `PascalCase` pour types. Pour un couple contrat/implémentation, choisissez une convention et tenez-vous-y : `IService/ServiceImpl`, ou `Service/ServiceImpl`.
- **Séparer le servent du lanceur** — la classe qui implémente l'interface distante et la classe qui contient le `main` sont deux responsabilités différentes. Le TD 1 sépare déjà `HelloImpl` de `Server`.
- **Une interface séparée de la logique** — la CLI exigée par le TP (étape A8) s'appuie sur une couche client qui, seule, manipule les interfaces distantes. Deux clients CLI lancés côte à côte font une démonstration bien plus parlante qu'un `main` figé (§ 3.6).
- **Commencez souvent** — un commit par fonctionnalité laisse une trace de votre démarche.

POUR ALLER PLUS LOIN

- CM 02, sections 1 à 5 — le modèle complet, avec traces d'exécution.
- TD 1 — lecture ligne à ligne de `helloWorld`.
- `snippets/hello-rmi-multi-project` — la structure de projets, en version exécutable.
- Annexe « Patrons utiles aux architectures distribuées » et `resources/design-patterns/observer` — le patron de l'étape A6 en version locale, et ce qui change à travers le réseau.
- Documentation officielle : [Java RMI API Guide, Release 21](#) et la javadoc de `java.rmi`.